Ders 26 Genişletilmiş Notu: Ölümcül Kilitlenme: Koşullar, Kaynak Atama Çizgesi, Önleme, Kaçınma, Banker Algoritması ve Tespit

Ders 26 Genişletilmiş Notu: Ölümcül Kilitlenme: Koşullar, Kaynak Atama Çizgesi, Önleme, Kaçınma, Banker Algoritması ve Tespit

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

İçindekiler

- 1. Dersin Planı ve Sistem Modeli
- 2. Ölümcül Kilitlenmenin Dört Koşulu
- 3. Muteksin Yanlış Kullanımıyla Kilitlenme
- 4. Kaynak Atama Çizgesi
- 5. Kilitlenme ile Başa Çıkma Yöntemleri
- 6. Kilitlenme Önleme
- 7. Kitaptaki İki Örnek: İki Thread ve Banka Transferi
- 8. Kilitlenmeden Kaçınma ve Güvenli Durum
- 9. Kaynak Atama Çizgesi Algoritması
- 10. Banker Algoritması
- 11. Kilitlenme Tespiti
- 12. Kilitlenmeden Kurtarma
- Örnek Sorular

1. Dersin Planı ve Sistem Modeli

Ders, Silberschatz'ın *Operating System Concepts* kitabının 7. bölümünden, ölümcül kilitlenme (deadlock) konusuyla devam etti. Ders Zoom üzerinden yapıldı.

Hoca önce dersin planını verdi. Önceki derslerde adı geçen deadlock'ın ne olduğu ve nasıl tanımlandığı konuşulacak. Sonra bir işletim sisteminin deadlock konusunda yapabileceği şeylere bakılacak:

- Deadlock'ın oluşmasını engellemek: önleme (deadlock prevention).
- Çeşitli tedbirler alarak deadlock'tan kaçınmak (deadlock avoidance).
- Deadlock oluşmasına izin verip, oluşunca onu tespit etmek (detection) ve sistemi kurtarmak (recovery).
- Hocanın şaka yollu "en sevdiğimiz" dediği yöntem: Deadlock ile ilgili hiçbir şey yapmamak. "Kim çözsün? Kullanıcı çözsün."

1.1 Kaynaklar

Bir bilgisayar sisteminde kaynaklar (resources) vardır. Kaynaklar fiziksel ya da mantıksal (logical) olabilir.

- **Fiziksel kaynaklar:** İşlemci zamanı, bellek kullanımı, G/Ç aygıtları (I/O devices). Çevre birimleri de birer kaynaktır: yazıcı (printer), CD-ROM, USB aygıtı.

- **Mantıksal kaynaklar:** Dosya sistemindeki bir dosyaya erişim, senkronizasyon bölümünde konuşulan semaforlar ve muteksler (mutex).

1.2 Kaynak Kullanım Sırası

Sistem çalışırken bir süreç bir kaynağı şu sırayla kullanır:

1. **İstek (request):** Süreç kaynağa ihtiyacı olduğunu belirtir.
2. **Kullanım (use):** Süreç kaynağı kullanır.
3. **Bırakma (release):** Süreç işi bitince kaynağı boşa çıkarır.

Hocanın örneği bellek ayırmaydı. Programlama derslerinde `malloc` ya da `calloc` ile bir bellek ayırdığınızda, işiniz bittikten sonra bu belleği `free` ile bırakmanız gerekir. Birazdan görülecek önleme, kaçınma ve tespit algoritmalarında kaynaklar ve süreçler bir çizge (kaynak atama çizgesi) ile modellenecek. Bu modelleme de bu istek, kullanım ve bırakma yapısına dayanır.

2. Ölümcül Kilitlenmenin Dört Koşulu

Deadlock'ın dört gerekli koşulu vardır. Hoca bu dört koşulun önemli olduğunu söyledi. Çünkü önleme yapılacaksa bu dört koşuldan birine "saldırılır".

> **Hocanın vurgusu:** "En iyi savunma saldırıdır." Deadlock için dört koşulun dördü birden gerekir. Bu koşullardan birinin oluşması engellenirse deadlock oluşamaz.

2.1 Karşılıklı Dışlama (Mutual Exclusion)

Kaynaklar paylaşılabilir (shareable) olabilir ya da yalnız tek bir sürecin kullanabileceği, karşılıklı dışlamalı (mutually exclusive) kaynaklar olabilir.

Hocanın örneği bir dosyaydı. Dosyaya okuma yönünde erişilecekse dosya paylaşılabilir bir kaynaktır. Yüzlerce süreç aynı anda bu dosyayı okuyabilir. Okuma işlemi dosyada bir değişikliğe neden olmadığı için burada karşılıklı dışlamadan söz edilemez. Ama erişim yazma yönündeyse durum değişir. Muteks gibi senkronizasyon için oluşturulmuş bir yapı da aynı anda birden çok süreç tarafından kullanılamaz; bu zaten doğasına aykırıdır. Deadlock için en az bir kaynağın bu şekilde paylaşılamaz olması gerekir.

2.2 Tut ve Bekle (Hold and Wait)

Bir grup sürecin her biri bir ya da daha fazla kaynağı elinde tutar. Aynı zamanda, o kümedeki başka süreçlere atanmış kaynakları kullanmak ister. Örneğin P1, P2, P3, P4 süreçlerinin her biri, diğerinin ihtiyaç duyduğu bir kaynağa sahip olmalıdır.

2.3 Kesme Yok (No Preemption)

Hoca burada bir karışıklığa dikkat çekti. Buradaki "preemption", süreç zamanlamasında konuşulan sürecin kesilmesi değildir. Kaynağın süreçten zorla geri alınması kastedilir.

Bugünkü yapıda süreç bir kaynağa ihtiyaç duyunca onu ister ve alır. İstek karşılanamazsa, kaynağın türüne göre süreç ya bekler ya da işinden vazgeçer:

- Örneğin 1 MB bellek ayırmak istediniz ve ayıramadınız. Ya işinize devam edersiniz ya da daha sonra denemek üzere şimdilik o işten vazgeçersiniz.
- Muteks kilidini almak istediyseniz, kilit müsait olana kadar işletim sistemi sizi bekletir.

Hocanın örneği dosya kopyalamaydı. Bir dosyadan başka bir dosyaya kopyalama yapacaksınız. Kaynak dosyayı okuma yönünde açtınız. Hedef dosyayı yazma yönünde açacaksınız ama o dosyaya zaten yazan birisi var. Beklemeye başlarsınız. Beklerken elinizdeki açık kaynak dosyayı bırakmazsınız. Kaynağı bırakmanın tek yolu, sürecin o şekilde yazılmış olmasıdır. Örneğin ikinci dosya hemen açılamazsa iki kaynağı birden bırakabilirsiniz. Ama bugünkü programlama modellerinde iş genelde böyle yürümez: Bir kaynağı alırsınız ve o kaynak elinizdeyken başka bir kaynağı beklersiniz.

2.4 Döngüsel Bekleme (Circular Wait)

Bir grup süreç arasında dairesel bir bekleme olmalıdır. Örneğin $\{P_0, P_1, \ldots, P_n\}$ kümesinde P_0, P_1'i bekler; P_1, P_2'yi bekler; ...; P_n de P_0'ı bekler.

Hoca numaralandırmaya takılmamak gerektiğini söyledi. Süreçlerin adlarının sıralı olması gerekmez. Bekleme zinciri P4, P2, P20, P17, P1 diye de gidebilir. Önemli olan, verilen süreç kümesi içinde kapalı, dairesel bir bekleme yapısının oluşmasıdır.

Bu dört koşul kritiktir. Herhangi biri yoksa deadlock'tan söz edilemez. Dördü birden olmalıdır.

3. Muteksin Yanlış Kullanımıyla Kilitlenme

Hoca burada çok karıştırılan bir noktaya değinmek istedi ve sınıfa sorular sordu.

Soru: Muteks kilidini ne için kullanıyoruz?

Cevap: Paylaşılan kaynağı korumak için. Senkronizasyon sağlamaya çalışıyoruz. Paylaşılan kaynak, özellikle kesmenin olduğu yerde, doğru korunmazsa bir yarış durumu (race condition) oluşur ve veri tutarsızlığı (data inconsistency) ortaya çıkabilir.

Soru: Muteks kilidini kullanırken yaptığınız işin düzgün olmasından kim sorumlu?

Cevap: Önce "işletim sistemi" cevabı geldi, sonra doğru cevap verildi: kullanıcı, yani programcı sorumludur.

Peki muteks düzgün kullanılmazsa ne olur? Bu, muteksler konuşulurken de konuşulmuştu. Bir işi yapmak için iki ayrı kaynağa ihtiyacınız olsun. Thread'lerinizden biri önce birinci kaynağın muteksini, sonra ikinci kaynağın muteksini kilitliyor. Diğer thread ise yanlış yazılmış: Önce ikinci kaynağın, sonra birinci kaynağın muteksini kilitliyor.

Senaryo şöyle gelişebilir:

1. Birinci thread çalışır, birinci muteksi alır. İkinciyi alamadan kesilir (preempt edilir).
2. İkinci thread çalışır. Önce ikinci muteksi alır. Sonra birinciyi almak ister ama birinci muteks birinci thread'dedir; bekler.
3. Birinci thread'e dönülür. O da ikinci muteksi almaya çalışır ama ikinci muteks ikinci thread'dedir; bekler.

Artık iki thread de kımıldayamaz. Dört koşul kontrol edilirse:

- **Döngüsel bekleme:** Birinci thread ikinciyi, ikinci thread birinciyi bekliyor. "Alın size daire."
- **Kesme yok:** Kaynak geri alınamıyor. İkinci thread "ikinci muteksi aldım ama birinciyi alamıyorum, o zaman ikinciyi bırakayım" demiyor.
- **Tut ve bekle:** Thread'ler kilitleri birer birer istiyor, ikisini birden istemiyor.
- **Karşılıklı dışlama:** Muteksin doğası gereği var.

Dört koşul da sağlandığı için deadlock oluşur.

> **Hocanın vurgusu:** Bu deadlock her çalıştırmada illa ve kesin olacak diye bir şey yoktur. "Kötü ve pis tarafı" da budur. Çok thread'li, çok süreçli yapılarda hata ayıklamanın (debugging) daha zor olmasının bir nedeni budur.

4. Kaynak Atama Çizgesi

Bir sistemde deadlock olup olmadığını anlamak ya da en azından sistemi modelleyip incelemek için kaynak atama çizgesi (resource-allocation graph, RAG) kullanılır.

4.1 Düğümler ve Kenarlar

Çizgede iki tür düğüm (vertex) vardır, çünkü modellenmeye çalışılan şey süreçler ve kaynaklardır:

- **Süreç düğümleri:** $P_1, P_2, P_3, \ldots$ Kitapta ve birçok yerde yuvarlak (daire) ile gösterilir.

- **Kaynak düğümleri:** $R_1, R_2, R_3, \ldots$ Kare ile gösterilir. Bir kaynağın birebir aynı (identical) birden çok kopyası (instance) varsa, karenin içine kopya sayısı kadar nokta ya da küçük kare çizilir.

Kenarlar (edges) oklardır. Okun anlamı yönüne bağlıdır:

- **İstek kenarı (request edge):** $P_i \to R_j$. P_i süreci R_j kaynağını kullanmak istiyor. Şu an ona sahip değil.
- **Atama kenarı (assignment edge):** $R_j \to P_i$. R_j kaynağı P_i sürecine atanmış, süreç onu kullanıyor. Kaynağın birden çok kopyası varsa ok, karenin içindeki noktadan (kopyadan) sürece çizilir.

4.2 Birinci Örnek: Döngü Yok

Slayttaki ilk örnekte üç süreç (P1, P2, P3) ve dört kaynak (R1, R2, R3, R4) var. R1 ve R3'ten birer kopya, R2'den 2, R4'ten 3 kopya var. Kenarlar şöyle:

- R2'nin bir kopyası P1'e atanmış. P1, R1'i istiyor.
- R1, P2'ye atanmış. R2'nin diğer kopyası da P2'de. P2, R3'ü istiyor.
- R3, P3'e atanmış.
- R4'ü kullanan kimse yok.

Soru: Bu çizgeye bakınca doğrudan ne söyleyebilirsiniz?

Sınıftan "hiçbir şey söyleyemeyiz" cevabı geldi. Hoca bunu kabul etti, sonra durumu açıkladı. P3'ün çalışması için gereken tek kaynak R3'tür ve onu zaten almıştır. P3 bir yerde çalışıp biter ve R3'ü bırakır. Bu durumda P2 istediği R3'ü alabilir. P2'nin diğer ihtiyaçları da karşılanmıştır. P2 çalışır ve sistemden çıkar. Çıkarken R1'i de bırakır. Bırakır bırakmaz P1 ile R1 arasındaki istek oku atama okuna döner, yani yönü ters çevrilir. Böylece P1 de çalışıp çıkar. Bu çizgede deadlock oluşturacak bir şey yoktur.

4.3 İkinci Örnek: Döngü ve Deadlock

Çizge biraz değiştirilir: P3 de R2'yi ister. Artık P3'ün çalışması için de bir kaynak gerekiyor ve o kaynakta şu an boş kopya yok.

Hocaya göre bir RAG'de deadlock olup olmadığını söylemenin en kolay yolu, çizgede bir döngü (cycle) aramaktır.

Soru: Bu çizgede kaç tane dairesel durum görüyorsunuz?

Cevap: İki tane.

1. $P_1 \to R_1 \to P_2 \to R_3 \to P_3 \to R_2 \to P_1$: P1, R1'i istiyor; R1, P2'ye atanmış; P2, R3'ü istiyor; R3, P3'e atanmış; P3, R2'yi istiyor; R2, P1'e atanmış. Hoca "çok çembere benzemiyor biliyorum ama dairesel bir durum var" dedi.
2. $P_2 \to R_3 \to P_3 \to R_2 \to P_2$: P2, R3'ü istiyor; R3, P3'e atanmış; P3, R2'yi istiyor; R2'nin diğer kopyası P2'ye atanmış.

Biri küçük, biri büyük iki döngü vardır. R2'nin iki kopyası da P1 ve P2'dedir. Hiçbir süreç ilerleyemez; bu örnekte deadlock vardır.

Hocanın genel kuralı şudur:

- Çizgede bir döngü varsa deadlock **olabilir**. Olması garanti değildir.
- Çizgede döngü yoksa, deadlock olmadığı "gönül rahatlığıyla" söylenebilir.

4.4 Üçüncü Örnek: Döngü Var, Deadlock Yok

Peki her döngü gördüğümüzde deadlock var mı? Bunu ayırt etmek için hoca üçüncü örneği gösterdi:

- P1, R1'i istiyor. R1'in iki kopyası farklı süreçlere atanmış: biri P2'ye, biri P3'e.
- P3, R2'yi istiyor. R2'nin bir kopyası P1'e, diğeri P4'e atanmış.

Çizgede $P_1 \rightarrow R_1 \rightarrow P_3 \rightarrow R_2 \rightarrow P_1$ döngüsü vardır. Döngü olduğu için deadlock olabilir deriz.

Soru: Burada deadlock var mı?

Cevap: Yok.

Çünkü sistemin bu durumdan kurtulabileceği bir çalışma sırası vardır. Örneğin P2 çalışır ve bir zaman R1'i bırakır. P1'in istek kenarı atama kenarına döner, P1 çalışıp çıkar. P1 çıkınca R2'yi bıraktığı için P3'ün R2'ye erişmesinin önünde engel kalmaz. Ya da P4 çalışır, R2'yi bırakır, R2 P3'e atanır, P3 çalışıp çıkar. Her durumda döngü çözülür.

4.5 Özet Kural

- Çizgede hiç döngü yoksa deadlock yoktur.
- Döngü varsa ve kaynakların her biri tek kopyalıysa deadlock kesin vardır.
- Döngü varsa ve kaynakların birden çok kopyası varsa deadlock ihtimali vardır.

Bu ihtimal şöyle kontrol edilir: Sistemin çalışıp o durumdan çıkabileceğini gösteren bir çalışma sırası (sekans) var mı diye bakılır.

> **Ders sonrası not:** Hoca "grafınız içerisindeki bütün kaynakların her biri tek kopyaysa kesin deadlock var" dedi. Bunun için döngüdeki kaynakların tek kopyalı olması yeterlidir; çizgedeki bütün kaynakların tek kopyalı olması gerekmez.

5. Kilitlenme ile Başa Çıkma Yöntemleri

İşletim sistemlerinde deadlock durumunu ele almak için farklı yöntemler önerilmiştir:

1. **Önleme (prevention):** Deadlock'ın dört temel koşulundan birinin oluşmasını engellemek. Birini engellerseniz deadlock'ın oluşmasını engellemiş olursunuz.
2. **Kaçınma (avoidance):** Hoca bunu bir tür ön hazırlık (provisioning) yapmaya benzetti. Deadlock'ı doğrudan engellemek yerine, sistem çalışırken süreçlerden ek bilgi istenir. Bu bilgiyle süreçlerin deadlock yaratmasından kaçınılmaya çalışılır.
3. **Tespit ve kurtarma (detection and recovery):** Sistemi deadlock oluşup oluşmamasından bağımsız bırakıp çalışmasına izin vermek. Deadlock oluşunca da buradan kurtulmak.
4. **Görmezden gelmek:** Bugünkü modern işletim sistemlerinin neredeyse tamamı deadlock konusunda bir şey yapmaz. Sistemin deadlock'a girmesini engelleyecek ek bir mekanizma kullanmaz.

Hoca bu genel kuralın bir istisnasından söz etti: lock'ların kullanımını, özellikle kilitlerin hangi sırayla alındığını denetleyen "Witness" adlı bir modül. Kilit sırasında bir tuhaflık varsa bir istisna (exception) üretir. Deadlock için en fazla düşünülen ve yapılan kısım budur. Onun dışında Windows'ta da Linux'ta da sistem çalıştırılır; deadlock'ın oluşması da, oluşursa çözülmesi de kullanıcıya kalır. Sistem "bir deadlock var, bunu tespit edeyim, temizlemek için ne yapmam gerekir" kısmıyla da çoğu zaman uğraşmaz.

> **Ders sonrası not:** Hoca Witness'ın Linux içinde bir modül olduğunu söyledi. Witness, büyük olasılıkla FreeBSD çekirdeğindeki kilit sırası denetleyicisidir. Linux'taki benzer araç büyük olasılıkla `lockdep`'tir (lock dependency validator).

6. Kilitlenme Önleme

Önlemede dört koşulun her birine ayrı ayrı "saldırmak" denenir.

6.1 Karşılıklı Dışlama

Bir kaynağın karşılıklı dışlama olmadan kullanılabilmesini sağlamak gerekir. Bu her zaman mümkün değildir. Örneğin muteks kilidi gibi bir yapı için "karşılıklı dışlama olmasın" demek anlamlı da değildir, mümkün de değildir. Paylaşılabilir kaynaklar için bu mümkündür ama bütün kaynakların paylaşılabilir olması düşünülemez. Bu koşul için yapılabilecek çok bir şey yoktur.

6.2 Tut ve Bekle

Süreç bir kaynağa ihtiyaç duyunca onu söylüyor, alıyor, sonra diğer kaynağa geçiyor. Bu bozulabilir. Hocanın örneği şu işti: CD-ROM'dan bilgi okunacak, diske yazılacak, diskte sıralanacak, sonra dosya yazıcıya gönderilecek.

Normal şartlarda iş şöyle yapılır:

1. CD-ROM'dan okumak için CD-ROM aygıtına erişilir.
2. Yazılacak dosya açılır. İki kaynak birlikte kullanılarak yazma yapılır.
3. Yazma bitince çoğu programlama dilinde aygıt kapatılır. Dosya okuma modunda açılır ya da yazma modundayken başa sarılıp okunur. Bu durumda dosyayı ilk açılışta okuma-yazma (read-write) modunda açmak gerekebilir.
4. Dosya satır satır okunur, yazıcı açılır ve çıktıya gönderilir.

Tut ve bekle koşulunu bozmanın iki yolu vardır:

- **Bütün kaynakları baştan istemek:** Süreç çalışmaya başlamadan önce ihtiyaç duyacağı bütün kaynakları işletim sistemine bildirir ve süreç başlarken hepsi ona atanır. Sorun: Daha CD-ROM'dan dosyaya kopyalama yapılırken hiç kullanılmayan yazıcı meşgul edilir ve başkasının kullanması engellenir. Yazıcıya nispeten yavaş yazılırken de CD-ROM hâlâ bloke edilir.
- **Elinde kaynak yokken istemek:** Süreç yalnız elinde hiç kaynak yokken kaynak isteyebilir. Aynı iş için önce CD-ROM ve dosya istenir, iş yapılır, ikisi de bırakılır. Elinde hiç kaynak kalmayınca dosya ve yazıcı istenir.

Hangisi yapılırsa yapılsın, çok verimli bir yapı olmaz. Zaman zaman açlığa (starvation) bile yol açabilir.

6.3 Kesme Yok

Süreç aldığı kaynağı bırakmıyor, tutmaya devam ediyordu. Bu özelliğe saldırmak için şu kural uygulanabilir: Elinde kaynak olan bir süreç bir kaynak daha ister ve isteği hemen karşılanmazsa, elindeki kaynakları da bırakır. Örnekte CD-ROM'u açtınız, diskteki dosyaya yazmak istediniz ama şu an izin verilmedi. O zaman CD-ROM'u da bırakırsınız. Bırakılan CD-ROM yeniden sürecin ihtiyaç listesine eklenir; süreç başa dönmek zorunda kalır.

Hocanın gösterdiği sıkıntı şudur: Birden çok, hatta batch süreçleriniz çalışıyor. Bir süreç çalışmış, çalışmış, son bir kaynağa gelmiş. O kaynağı da kullanıp işini bitirecek. "Olmaz, veremiyoruz kaynağı, diğer bütün kaynaklarını bırak" derseniz, belki de yaptığı bütün hesaplamaları iptal edersiniz. Kullanım oranı (utilization) "yerle bir olur". O ana kadar harcanan CPU zamanı boşa gider. Çok tercih edilecek bir yöntem değildir.

6.4 Döngüsel Bekleme

Süreçler arasında halka şeklinde bir bekleme olmalıydı. Bunu çözmek için kaynaklar numaralandırılabilir (enumerate). Örneğin bir dosyaya erişim kaynağına bir numara, bir diske başka bir numara, yazıcıya başka bir numara verilir. Hocaya göre bu çoğu zaman kaynağı bir fonksiyona, bir hash'e sokarak yapılabilir.

Numaralandırma şunu sağlar: Herkesin kaynakları artan sırayla istemesi şart koşulabilir. Bu şart konunca döngüsel bekleme otomatik olarak ortadan kalkar.

Bunun da uygulamada sıkıntıları vardır:

- Ek bir yük getirir.
- Programcının sıraya uyup uymadığını kontrol etmesi gerekir.
- Sistemden sisteme değişebileceği için taşınabilirlik (portability) sorunu vardır.

Bu yüzden kolay uygulanabilecek bir yöntem değildir.

> **Ek bilgi (derste anlatılmadı):** Artan sırayla isteme kuralının döngüyü neden kırdığı kısaca şöyle gösterilebilir. Bir döngüde her süreç, elindeki kaynaktan daha büyük numaralı bir kaynağı bekler. Döngü kapanırsa numaralar zincir boyunca hep artmış olur ve sonunda bir kaynağın kendi numarasından büyük olması gerekir. Bu imkânsız olduğu için döngü kurulamaz.

7. Kitaptaki İki Örnek: İki Thread ve Banka Transferi

Hoca bu kısmın daha çok teorik olarak konuşulduğunu söyledi. Slaytlarla kitabın içeriğinin tam tutmadığını, 9. baskının bazı slaytlarının kaydığını da ekledi.

7.1 İki Thread, İki Muteks

Slaytta iki thread vardı. Birincisi önce `first_mutex`, sonra `second_mutex` kilitliyor. İkincisi önce `second_mutex`, sonra `first_mutex` kilitliyor. Bu yapıda birkaç senaryo mümkündür:

- Birinci thread çalışıp biter, sonra ikincisi çalışıp biter. Her şey normaldir.
- Birinci thread iki kilidi de alır, işini yaparken kesilir. İkinci thread çalışır, kilidi istediği yerde tıkanır ve bekler. Birinci thread işini bitirince ikinci thread çalışır. Bu da sorunsuzdur.
- Birinci thread birinci kilidi alır ve kesilir. İkinci thread ikinci kilidi alır ve kesilir. Sonrası deadlock'tur.

7.2 Banka Transferi

Aynı şeyin farklı bir senaryosu: Bir `transaction` fonksiyonu var. Buradaki transaction veritabanı işlemi değil, parasal bir işlemdir; bir hesaptan diğerine para aktarılır. Fonksiyon şöyle çalışır:

1. Para çekilecek hesabın (`from`) kilidi alınır.
2. Paranın aktarılacağı hesabın (`to`) kilidi alınır.
3. Bir hesaptan para çekilir (`withdraw`), diğerine yatırılır (`deposit`).
4. Kilitler alınma sırasının tersiyle bırakılır.

Bu fonksiyon iki ayrı işlem için aynı anda çalıştırılırsa ne olur? Örneğin biri A'dan B'ye 25 dolar, diğeri B'den A'ya 50 dolar aktarıyor. Sınıftan "hiçbir şey olmaz, her şey normal olur" cevabı geldi. Oysa birinci işlem A'nın kilidini, ikinci işlem B'nin kilidini alıp kesilirse, ikisi de diğerinin elindeki kilidi bekler. Bu, 7.1'deki iki thread örneğinin aynısıdır.

> **Hocanın vurgusu:** Bu fonksiyon deadlock'a sebep olabilir. Deadlock'a sebep olmayacak şekilde nasıl güncellenebileceği üzerine düşünün.

Bu sorunun çözümü derste verilmedi; "Örnek Sorular" bölümünde çözümü verildi.

8. Kilitlenmeden Kaçınma ve Güvenli Durum

8.1 Temel Fikir

Hoca kaçınmayı ağlardaki bağlantı yönelimli (connection-oriented) yapıya, devre anahtarlamaya (circuit switching) benzetti. Hizmet kalitesi (quality of service) sağlanmaya çalışılırken "bir uçtan bir uca giderken bana 1 megabitlik bir hat lazım" denir. Karşı taraf bakar: Toplam 10 megabitin 9 megabitini kullanıyordur, sizin istediğiniz 1 megabittir. "Bu hattı sağlayabilirim" deyip karar verir.

Kaçınmada da benzer bir yapı kurulur. Her sürecin kullanacağı her kaynak türünden en fazla (maksimum) kaç tane kullanacağını önceden söylemesi beklenir. Süreçlerin beyan ettiği bu bilgiye göre, sistem modeli üzerinde sürekli olarak sistemin güvenli (safe) mi güvensiz (unsafe) mi olduğu anlaşılmaya çalışılır.

Sistemin durumuna kaynak atama durumu (resource-allocation state) denir. Bu durum şunlarla belirlenir: Şu an hangi sürece hangi kaynaktan kaç tane atandığı, kaç kaynağın boşta olduğu ve süreçlerin en fazla ihtiyaçları. Bu bilgiyle herhangi bir T anında sistemin kilitlenmesine engel olunup olunamayacağına bakılır.

8.2 Güvenli ve Güvensiz Durum

Sistem ya güvenli ya da güvensiz durumdadır. Güvensiz durumda her an deadlock olmak zorunda değildir, ama deadlock oluşma potansiyeli vardır.

> **Ders sonrası not:** Hoca burada "safe state değilse zaten deadlock olmaz" dedi. Doğrusu: Sistem güvenli durumdaysa deadlock olmaz. Güvensiz durumda deadlock olabilir ama olmak zorunda değildir.

Güvenli durumun tanımı, RAG konuşulurken söylenen şeyin aynısıdır. Sistemde süreçler, kaynaklar, bu kaynaklara yapılmış mevcut atamalar ve süreçlerin daha isteyebileceği kaynaklar vardır. Sistemin T_0 anındaki durumundan itibaren bir çalışma sırası oluşturulur. Süreçler bu sırayla, ihtiyaçlarına göre çalıştırılınca hepsi başarıyla tamamlanabiliyorsa sistem güvenli durumdadır. Böyle bir sıraya güvenli sıra (safe sequence) denir.

Hoca buradaki bütün formüllerin özetinin buna çıktığını söyledi. Yapılan iş şudur:

1. Mevcut boş (available) kaynaklarla, elindekilerin üstüne en fazla ihtiyacını verebileceğimiz bir süreç aranır.
2. Böyle bir süreç bulunursa çalışır ve tamamlanınca tuttuğu bütün kaynaklar geri döner.
3. Süreçler adım adım çalıştırılarak hepsi bitirilmeye çalışılır. Bu sırada bir sıra bulunabiliyorsa sistem güvenli durumdadır.

Kaçınma için çeşitli algoritmalar vardır. Derste iki temel algoritma incelendi:

- Her kaynaktan tek kopya varsa: kaynak atama çizgesi algoritması.
- Kaynakların birden çok kopyası varsa: banker algoritması.

Bunların dışında başka yaklaşımlar da kurulabilir.

9. Kaynak Atama Çizgesi Algoritması

RAG'de iki tür ok vardı: kaynaktan sürece atama kenarı, süreçten kaynağa istek kenarı. Kaçınma için RAG kullanılacaksa bir üçüncü kenar türü gerekir: talep kenarı (claim edge). Bu da süreçten kaynağa çizilir ama kesikli (hocanın deyişiyle "tırtıklı") bir ok olarak: $P_i \dashrightarrow R_j$.

Kaçınmada süreçler bütün ihtiyaçlarını önceden bildiriyordu. Talep kenarı bunu gösterir: Süreç şu an bu kaynağı kullanmıyor ve istemiyor, ama ileride isteyebilir. Kenarın yaşam döngüsü şöyledir:

1. Süreç kaynağı isteyince kesikli ok düz hâle gelir; istek kenarı olur.
2. İstek kabul edilince yönü değişir; atama kenarı olur.

9.1 Slayttaki Örnek

- R1, P1'e atanmış.
- R2, P2'ye atanmış.
- P2, R1'i istiyor (istek kenarı).
- P1'den R2'ye bir talep kenarı var: P1 ileride R2'yi isteyecek.

Bu duruma bakınca sistem güvenli durumda denemez; güvensiz durumdadır. Güvensiz durumdan çıkabilir de, çıkamayıp deadlock'a da girebilir. P1 R2'yi isterse R2 P2'nin elindedir, P1 onu alamaz. P2 de R1'i istiyor ama R1 P1'in elindedir. Döngüsel bekleme oluşur, yani deadlock.

> **Ders sonrası not:** Hoca önce "deadlock'a girmesi için P2'nin R2'yi istiyor olması gerekir" dedi. R2 zaten P2'dedir. Deadlock'a yol açacak istek, P1'in R2'yi istemesidir. Hoca hemen ardından durumu doğru anlattı.

Ama bu yapıda P1'in ilerleyişi bilinmiyor; yalnız en fazla ne isteyeceği biliniyor. P1, R2'yi isteyeceği sırada R1'i bırakmış olabilir. O zaman R1 P2'ye atanır, P2 çalışıp çıkar, sonra P1 çalışıp bütün işini bitirir. Bu yüzden şu an "güvenli durum" denemez, ama "deadlock var" da denemez.

P1, R2'yi istediği zaman deadlock oluşacağı bilinmektedir. Kaçınma yapıldığı için, bu talep kenarı istek kenarına dönüşürse istek onaylanmaz. Kural şudur: Bir istek, atama kenarına dönüştüğünde çizgede döngü oluşacaksa karşılanmaz.

RAG üzerinden birden çok kopyası olan kaynaklar için değerlendirme yapmak pek mümkün değildir. Bunun için banker algoritması kullanılır.

10. Banker Algoritması

10.1 Adı ve Şartları

Adının nedeni "dümdüz" bir nedendir: Bankalar için kullanılan bir yapıdır. Bir banka bütün müşterilerinin nakit ihtiyacını karşılayabilecek durumda olmalıdır. Karşılayamazsa batar. Ad buradan gelir.

> **Ek bilgi (derste anlatılmadı):** Banker algoritması büyük olasılıkla Edsger W. Dijkstra tarafından 1960'ların ortasında, THE çoklu programlama sistemi üzerinde çalışırken önerilmiştir.

Algoritmanın şartları:

- Her süreç, en fazla kullanacağı kaynakları önceden (a priori) bildirmelidir.
- Bildirimden sonra sistemin durumu sürekli kontrol edilir. Gelen isteklerin karşılanıp karşılanamayacağına ve karşılanırsa deadlock ihtimali doğup doğmayacağına bakılır. Bu yüzden bir süreç istekte bulunduğunda istek her zaman karşılanmayabilir; süreç bekletilebilir.

  > **Hocanın vurgusu:** Her istek geldiğinde, istek karşılanırsa sistemin ne hâle geleceği kontrol edilmelidir. Bu yüzden banker algoritması doğrudan uygulanabilir bir yapı değildir. Her istekte sistemi yeniden kontrol etmek, algoritmanın karmaşıklığı düşünülünce oldukça yoğun işlem gücü demektir.

10.2 Veri Yapıları

n süreç ve m kaynak türü olsun.

- **`Available`:** m uzunluğunda bir dizi. Her göz, o kaynak türünden şu an kaç tane boş kopya olduğunu gösterir.
- **`Max`:** $n \times m$ matris. $Max[i][j] = k$ ise P_i süreci R_j kaynağından en fazla k tane ister.
- **`Allocation`:** $n \times m$ matris. Herhangi bir T anında hangi kaynaktan kaç tanesinin hangi sürece atandığını gösterir.
- **`Need`:** $n \times m$ matris. Sürecin daha ne kadar isteyebileceğini gösterir. En fazla ne kullanacağı ve şu an ne kullandığı bilindiği için:

$$Need[i][j] = Max[i][j] - Allocation[i][j]$$

10.3 Güvenlik Algoritması

Banker algoritmasının bir güvenlik (safety) kısmı vardır: Sistem şu an güvenli durumda mı?

1. `Work`, `Available`'ın bir kopyası olarak oluşturulur: `Work = Available`. `Finish` adlı bir dizi oluşturulur ve başlangıçta bütün i için `Finish[i] = false` yapılır; hiçbir süreç bitmemiştir.
2. `Finish[i] == false` ve `Need[i]` ≤ `Work` olan bir i aranır; yani ihtiyacı eldeki kaynaklardan az ya da eşit olan, henüz bitmemiş bir iş. Yoksa 4. adıma geçilir.
3. `Work = Work + Allocation[i]` ve `Finish[i] = true` yapılır. 2. adıma dönülür.
4. Bütün i için `Finish[i] == true` ise durum güvenlidir.

Hoca 3. adımı şöyle açıkladı: O sürece `Need`'i kadar kaynak verilir, süreç çalışır ve biter. Bitince bütün kaynaklarını geri verir. Bu yüzden onun `Allocation`'ı boş kaynakların üstüne konur. Ama bu işlem gerçek `Available` üzerinde değil, geçici bir yapı olan `Work` üzerinde yapılır. Bu adımlar tekrarlanarak herkesin işi görülebiliyorsa, yani bir sıra bulunabiliyorsa durum güvenlidir.

10.4 İstek Algoritması

Sistemin güvenli olup olmadığını anlamak yetmez. Bir de bir isteğe karşılık kaynağın atanıp atanmayacağına karar veren yapı gerekir. P_i süreci `Request[i]` ile gelsin:

1. `Request[i]` ≤ `Need[i]` olmalıdır. Değilse hata vardır: Süreç, söylediğinden farklı (fazla) bir şey istiyordur. Bu durumda istek zaten karşılanmaz.

2. `Request[i]` ≤ `Available` olmalıdır. Mevcut kaynaklarla istek karşılanamıyorsa süreç bekletilir.
3. İstek karşılanabiliyorsa, karşılanmış gibi yapılar güncellenir:
   - `Available = Available - Request[i]`
   - `Allocation[i] = Allocation[i] + Request[i]`
   - `Need[i] = Need[i] - Request[i]`

   Sonra güvenlik algoritması çalıştırılır. Sistem güvenliyse istek karşılanır. Değilse istek karşılanmaz: Eski duruma dönülür ve süreç bekler.

10.5 Slayttaki Örnek

Beş süreç (P0-P4) ve üç kaynak türü var: A'dan 10, B'den 5, C'den 7 kopya. T_0 anındaki durum:

Süreç	Allocation (A B C)	Max (A B C)	Need (A B C)
P0	0 1 0	7 5 3	7 4 3
P1	2 0 0	3 2 2	1 2 2
P2	3 0 2	9 0 2	6 0 0
P3	2 1 1	2 2 2	0 1 1
P4	0 0 2	4 3 3	4 3 1

`Available` = (3, 3, 2). Toplam atanmış kaynak (7, 2, 5) olduğundan, (7, 2, 5) + (3, 3, 2) = (10, 5, 7) toplam kopya sayısını verir. `Need` sütunu $Max - Allocation$ ile hesaplanır.

Hoca güvenli sırayı şöyle buldu:

1. P1 çalıştırılabilir mi? P1 en fazla 3 tane A isteyecek, 2'sini kullanıyor. 1 A, 2 B, 2 C lazım: $(1, 2, 2) \leq (3, 3, 2)$. Var, çalışır. Bitince kaynaklarını bırakır: `Work` = (3, 3, 2) + (2, 0, 0) = (5, 3, 2). Hoca "A'nın sayısı 3'ten 5'e çıkacak" dedi.
2. P3: $(0, 1, 1) \leq (5, 3, 2)$. Çalışır. `Work` = (5, 3, 2) + (2, 1, 1) = (7, 4, 3). Hoca "A'ların sayısı 7, B 4, C 3 olacak" dedi.
3. P4: $(4, 3, 1) \leq (7, 4, 3)$. Çalışır. `Work` = (7, 4, 3) + (0, 0, 2) = (7, 4, 5).
4. P0: $(7, 4, 3) \leq (7, 4, 5)$. Çalışır. `Work` = (7, 4, 5) + (0, 1, 0) = (7, 5, 5).
5. P2: $(6, 0, 0) \leq (7, 5, 5)$. Çalışır. `Work` = (7, 5, 5) + (3, 0, 2) = (10, 5, 7).

Hoca böylece ⟨P1, P3, P4, P0, P2⟩ sırasını buldu ve T_0 anında sistemin güvenli durumda olduğunu söyledi. Slayttaki sıra ⟨P1, P3, P4, P2, P0⟩ idi. Hocanın dediği gibi "böyle ya da böyle çıkıyor": İki sıra da güvenlidir. ⟨P1, P3, P4, P2, P0⟩ sırasında ara `Work` değerleri (5, 3, 2), (7, 4, 3), (7, 4, 5), (10, 4, 7), (10, 5, 7) olur. İkisi de Python ile doğrulandı.

10.6 P1'in İsteği: (1, 0, 2)

Slaytta aynı örnek değiştirilir: P1, `Request` = (1, 0, 2) ile gelir.

1. $(1, 0, 2) \leq Need_1 = (1, 2, 2)$: Uygun.
2. $(1, 0, 2) \leq Available = (3, 3, 2)$: Uygun.
3. Yapılar güncellenir: `Available` = (2, 3, 0), $Allocation_1$ = (3, 0, 2), $Need_1$ = (0, 2, 0).

Güvenlik algoritması: P1'in ihtiyacı (0, 2, 0) eldeki (2, 3, 0) ile karşılanır; çalışır ve `Work` = (5, 3, 2) olur. Sonra kaynaklar geri döneceği için P3, sonra P4, sonra P0 ve P2 çalıştırılabilir. ⟨P1, P3, P4, P0, P2⟩ güvenli bir sıradır. İstek karşılanır. Python ile doğrulandı.

10.7 Ardından P4'ün İsteği: (3, 3, 0)

Sistem bu yeni durumdayken P4 (3, 3, 0) isteğiyle gelir. P4'ün `Need`'i (4, 3, 1) olduğu için bunları isteyebileceği bilinmektedir.

1. $(3, 3, 0) \leq (4, 3, 1)$: Uygun.
2. $(3, 3, 0) \leq Available = (2, 3, 0)$? Hayır. A'dan yalnız 2 tane vardır.

İstek karşılanamaz; P4 bekler.

10.8 Ardından P0'ın İsteği: (0, 2, 0)

1. P0'ın ihtiyaçlarından daha düşük bir istektir: $(0, 2, 0) \leq (7, 4, 3)$. Doğru bir istek.
2. Mevcutlarla karşılanabilir: $(0, 2, 0) \leq (2, 3, 0)$.
3. Karşılandıktan sonra sistemin güvenli olup olmadığına bakılmalıdır. (0, 2, 0) `Available`'dan düşülür ve `Available` = (2, 1, 0) olur. P0'ın `Allocation`'ı (0, 3, 0) olur.

Hoca burada "büyük ihtimalle deadlock olacak, ama siz çözüm bakın" deyip konuyu bıraktı. Güvenlik algoritması çalıştırılırsa (ara adım eklendi): Yeni `Need` değerleri P0: (7, 2, 3), P1: (0, 2, 0), P2: (6, 0, 0), P3: (0, 1, 1), P4: (4, 3, 1). `Work` = (2, 1, 0) ile hiçbir sürecin ihtiyacı karşılanamaz: P1 B'den 2 ister ama 1 vardır; P3 C'den 1 ister ama 0 vardır; diğerleri A'dan fazla ister. Durum güvensizdir. İstek karşılanmaz, eski duruma dönülür ve P0 bekler. Python ile doğrulandı; beş sürecin hiçbir sıralaması güvenli değildir.

> **Ders sonrası not:** Hoca "büyük ihtimalle deadlock olacak" dedi. Banker algoritması deadlock oluşmadan önce davranır: Yeni durum güvensiz çıktığı için istek reddedilir ve P0 bekletilir. Deadlock oluşmaz; yalnız güvensiz duruma girilmez.

11. Kilitlenme Tespiti

Üçüncü yöntem tespit ve kurtarmadır. Bir deadlock'ı ortadan kaldırabilmek için önce onu tespit etmek, sonra kurtarmak gerekir.

11.1 Tek Kopyalı Kaynaklar: Bekleme Çizgesi

Her kaynaktan bir tane varsa RAG, bekleme çizgesine (wait-for graph) çevrilebilir. Çevirirken RAG'deki kaynak düğümleri kaldırılır ve hangi sürecin hangi süreci beklediği çıkarılır: $P_i \to R \to P_j$ yolu, $P_i \to P_j$ kenarı olur. Bekleme çizgesinde bir döngü varsa yine "deadlock var" denir.

Slayttaki örnekte hocanın anlattıkları:

- P1, R2'yi almış (R2 P1'e atanmış). P1, R1'i almaya çalışıyor. R1, P2'ye atanmış. Bu yüzden P1'den P2'ye bir ok gerekir.
- P2, R3, R4 ve R5'i almaya çalışıyor. R3'ten P5'e giden atama kenarı yüzünden P2'den P5'e bir ok gerekir. R4 P3'e atanmış olduğu için P2'den P3'e bir ok gerekir.
- P3 ve P2 ikisi birden R5'i almaya çalışıyor. R5 P4'e atanmış. Bu yüzden P3'ten P4'e ve P2'den P4'e ok gerekir.
- P4'ten P1'e de bir ok vardır: P4, P1'in elindeki R2'yi bekler.

Bekleme çizgesinin kenarları: $P_1 \to P_2$, $P_2 \to P_3$, $P_2 \to P_4$, $P_2 \to P_5$, $P_3 \to P_4$, $P_4 \to P_1$.

Hoca "üçgen şeklinde ama aslında bir döngü var, bir halka var" dedi. Bu çizgede iki döngü vardır (Python ile doğrulandı):

- $P_1 \to P_2 \to P_4 \to P_1$ (hocanın gösterdiği üçgen)
- $P_1 \to P_2 \to P_3 \to P_4 \to P_1$

Döngü olduğu için burada deadlock vardır.

Her kaynağın tek kopyası olduğu bu durumda işler kolaydır; hocaya göre karmaşıklık $O(n^2)$'dir.

> **Ek bilgi (derste anlatılmadı):** n süreçli bir bekleme çizgesinde en fazla n^2 kadar kenar olabilir. Döngü arama, derinlik öncelikli arama (DFS) ile düğüm ve kenar sayısıyla doğru orantılı sürede yapılabildiği için karmaşıklık büyük olasılıkla bu yüzden $O(n^2)$ olarak verilir.

11.2 Çok Kopyalı Kaynaklar

Birden çok kopya olduğunda işler biraz karışır. Banker algoritmasına benzer bir yapı kurulur:

- **`Available`:** Kaynaklar bir diziyle ifade edilir. Dizinin her gözü o kaynaktan kaç tane boş olduğunu gösterir.
- **`Allocation`:** Hangi sürece hangi kaynaktan ne kadar atandığını gösteren matris.

- **`Request`:** Hangi sürecin hangi kaynaktan şu an ne kadar istediğini gösteren matris. Banker'daki `Need`'in yerini alır.

Algoritma şöyle çalışır: Bütün matris taranır. Bitmemiş işler arasında, eldeki kaynaklarla isteği karşılanabilecek bir süreç var mı diye bakılır. Varsa, o kaynaklar o sürece atanmış, süreç çalışmış ve bitmiş gibi düşünülür. Süreç bitince elindeki kaynakları da boşa çıkarır, yani eldeki kaynaklar artar. Sonra geri dönüp bitirilebilecek başka bir süreç aranır.

- Bu işlem tamamlandığında bütün süreçler tamamlanabilir hâle geliyorsa, yani bir çalışma sırası varsa, "deadlock yok" denir.
- Süreçlerden biri bile bitemiyorsa, yani bir i için `Finish[i] == false` kalıyorsa, sistemde deadlock vardır.

Buradaki sıkıntı, $n \times m$ bir matris üzerinde çalışılmasıdır.

> **Ders sonrası not:** Hoca karmaşıklığı "n çarpı m kare" olarak söyledi. Kitaptaki değer $O(m \times n^2)$'dir (m kaynak türü, n süreç): Her turda en fazla n süreç, her süreç için m karşılaştırma yapılır ve en fazla n tur gerekir.

11.3 Slayttaki Örnek

P0'dan P4'e beş süreç ve üç kaynak türü var: A'dan 7, B'den 2, C'den 6 kopya. Kaynakların hepsi atanmış durumdadır: `Available` = (0, 0, 0).

Süreç	Allocation (A B C)	Request (A B C)
P0	0 1 0	0 0 0
P1	2 0 0	2 0 2
P2	3 0 3	0 0 0
P3	2 1 1	1 0 0
P4	0 0 2	0 0 2

Hocanın çözümü:

1. P0'ın bir isteği var mı? Yok. Çalışır ve biter. Bitince B'den bir tane boşa çıkar: `Work` = (0, 1, 0).
2. P2'nin yeni bir isteği yok. O da çalışır ve çıkar: `Work` = (0, 1, 0) + (3, 0, 3) = (3, 1, 3).
3. P3'ün isteği (1, 0, 0), P2'nin boşalttığı kaynaklarla karşılanır: `Work` = (5, 2, 4).
4. Sonra P1: $(2, 0, 2) \leq (5, 2, 4)$; `Work` = (7, 2, 4).
5. Sonra P4: $(0, 0, 2) \leq (7, 2, 4)$; `Work` = (7, 2, 6).

⟨P0, P2, P3, P1, P4⟩ sırasıyla hepsi "tıkır tıkır" çalışabilir. Burada deadlock var denemez. Ara değerler Python ile doğrulandı (ara adım eklendi).

Değişiklik: P2'nin isteği (0, 0, 0) yerine (0, 0, 1) olsun; yani P2 C'den bir tane daha istesin. O zaman işler karışır. P0 doğrudan çalışabilir, zaten ek bir isteği yoktur. Ama P0'dan geri gelen kaynaklarla (`Work` = (0, 1, 0)) P2'yi çalıştırma şansı artık yoktur, çünkü C'den hiç boş kopya yoktur. P1, P3 ve P4'ün istekleri de karşılanamaz. Sistemde deadlock vardır.

> **Ders sonrası not:** Hoca deadlock'taki süreçleri "P2, P3, P4" olarak saydı. Python ile doğrulandığı üzere deadlock'taki süreçler P1, P2, P3 ve P4'tür. P1'in isteği (2, 0, 2) de eldeki (0, 1, 0) ile karşılanamaz.

11.4 Tespit Algoritması Ne Zaman Çalıştırılmalı?

Tespit algoritmasında da banker algoritmasındaki gibi bir sorun vardır. Deadlock olduğunu söyleyebilmek için algoritmanın çalışması gerekir. Peki ne zaman?

- **Her istek yapıldığında:** Banker algoritmasında konuşulduğu gibi sisteme fazladan çok yük getirir.
- **Periyodik olarak:** Yük biraz azalır. Ama iki çalıştırma arasında birden çok istek karşılanır ve sistem çalışmaya devam eder. Önceki durumla sonraki durum arasında birden çok değişiklik olabileceği için, tam olarak hangi isteğin deadlock'a neden olduğu bilinemez.

Bu neden önemlidir? Tespit boşu boşuna yapılmaz. Deadlock oluştuğu fark edilince çözülmek istenir. Çözmenin yolu, deadlock'a sebep olan süreci ortadan kaldırmaktır. Kimin sebep olduğu bilinmezse ortadan kaldırmak da sıkıntılı olur. Hoca, neyin sebep olduğu bilinse bile çözmek için farklı şeylerin denenebileceğini de ekledi.

12. Kilitlenmeden Kurtarma

Deadlock'a sebep olan süreç ortadan kaldırılırsa, tuttuğu kaynaklar boşa çıkar ve diğer süreçler ilerleyebilir. Ama deadlock genelde tek bir sürecin işi değildir. P1 tek başına deadlock yaratmaz; deadlock P1 ile P2 arasında oluşur. En basit hâlinde iki süreç birbirinin kaynağını ister ve bir döngü oluşur. Hangisi ortadan kaldırılacak?

12.1 Kurban Seçimi

Kurban (victim) seçiminde kullanılabilecek ölçütler:

- Sürecin önceliği (priority).
- O zamana kadar kullandığı CPU süresi.
- Yapması gereken işin ne kadarını tamamladığı.
- Kullandığı kaynaklar.
- Sonlandırılması gereken süreç sayısı: Deadlock yalnız iki süreç arasında olmayabilir. On sürecin birbirine girdiği, "zincirleme trafik kazası" gibi bir deadlock oluşabilir. Bu durumda en az sayıda süreci iptal ederek deadlock'ı çözmek bir yaklaşımdır.
- Sürecin çalışma şekli: Örneğin etkileşimli (interactive) bir sistemde, kullanıcıyla etkileşim hâlindeki bir süreç yerine arka plandaki (background) bir işi ortadan kaldırmak tercih edilebilir. Hocaya göre bu da bir açıdan önceliğe girer.

Her biri "içinden çıkılmaz" başka bir sorudur. Bütün bu alternatiflerde yapılan şey bir kurban seçmektir. Kurban, maliyeti en düşük olacak şekilde seçilir. Maliyetin bir kısmı şudur: Kurban seçilen süreç başladığı andan itibaren kullandığı işlemci zamanı çöpe atılmış olur.

12.2 Geri Alma ve Açlık

Ayrıca kurban, deadlock öncesindeki güvenli bir duruma geri alınmalı (rollback) ve oradan yeniden başlaması sağlanmalıdır.

Bu da yeni bir sorun getirir. Hoca şu senaryoyu anlattı: Öncelik tabanlı bir sisteminiz ya da etkileşimli işleriniz var, batch işleri ise sürekli kurban ediliyor. Bir batch işi başladı, çalıştı, çalıştı, deadlock oluştu; iptal edildi. Yeniden başa alındı. Geldi, tam bitirecekken yine deadlock oldu; yine atıldı. Hep aynı süreç kurban seçilirse iş açlığa (starvation) gider.

Bu yüzden deadlock'ı çözmeye çalışmak, arka tarafta düşünülmesi ve hesaba katılması gereken ciddi sıkıntılar içerir. Modern işletim sistemlerinde bu iş kullanıcıya bırakılır.

Örnek Sorular

Bu derste kayıt yalnız altyazıdan incelendi; aşağıdaki sorular derste sözlü sorulan sorulardır.

1. (derste sözlü, 00:21:33) Muteks kilidini ne için kullanırız? Muteksi kullanırken yapılan işin doğru olmasından kim sorumludur?

> **Çözüm:** Paylaşılan kaynağı korumak, yani senkronizasyon sağlamak için. Korunmazsa yarış durumu oluşur ve veri tutarsızlığı ortaya çıkabilir. Muteksin doğru kullanılmasından işletim sistemi değil, kullanıcı (programcı) sorumludur. Yanlış sırayla alınan kilitler deadlock'a yol açabilir (Bölüm 3).

2. (derste sözlü, 00:29:10) Üç süreç (P1, P2, P3) ve dört kaynak (R1 ve R3 tek kopya, R2 iki kopya, R4 üç kopya) var. R2'nin bir kopyası P1'de, P1 R1'i istiyor. R1 ve R2'nin diğer kopyası P2'de, P2 R3'ü istiyor. R3 P3'te. Bu kaynak atama çizgesine bakınca ne söyleyebilirsiniz?

> **Çözüm:** Çizgede döngü yoktur, deadlock yoktur. P3 bitip R3'ü bırakır, P2 R3'ü alıp biter ve R1'i bırakır, sonra P1 çalışıp biter.

3. (derste sözlü, 00:31:36) Önceki çizgede P3 de R2'yi isterse kaç tane dairesel durum oluşur?

> **Çözüm:** İki: $P_1 \to R_1 \to P_2 \to R_3 \to P_3 \to R_2 \to P_1$ ve $P_2 \to R_3 \to P_3 \to R_2 \to P_2$. R2'nin iki kopyası da P1 ve P2'de olduğu için hiçbir süreç ilerleyemez; deadlock vardır.

4. (derste sözlü, 00:35:36) P1 R1'i istiyor; R1'in iki kopyası P2 ve P3'te. P3 R2'yi istiyor; R2'nin iki kopyası P1 ve P4'te. Çizgede döngü var. Deadlock var mı?

> **Çözüm:** Yok. P2 ya da P4 bitince R1 ya da R2'nin bir kopyası boşalır ve döngü çözülür (Bölüm 4.4).

5. (derste sözlü, 00:50:31) Bir hesaptan diğerine para aktaran `transaction(from, to, amount)` fonksiyonu önce `from`'un, sonra `to`'nun kilidini alıyor. Aynı anda A'dan B'ye 25 dolar ve B'den A'ya 50 dolar aktarılırsa ne olabilir? Fonksiyon deadlock'a yol açmayacak şekilde nasıl yazılır?

Sınıftan "hiçbir şey olmaz" cevabı geldi. Doğrusu: Birinci işlem A'nın, ikinci işlem B'nin kilidini alıp kesilirse ikisi de diğerinin kilidini bekler; deadlock olur.

> **Çözüm (derste verilmedi):** Döngüsel bekleme koşulu kırılır: Kilitler her zaman aynı genel sırayla alınır, örneğin hesap numarası küçük olanın kilidi önce. Böylece iki ters yönlü transfer de önce aynı kilidi ister ve döngü kurulamaz. Aşağıdaki POSIX thread sürümü gcc ile derlendi ve iki thread'in her biri ters yönde 1.000.000 transfer yaptığında takılmadan bitti; toplam bakiye korundu. Kilitleri `from`, `to` sırasıyla alan özgün sürüm aynı testte kilitlendi (10 saniyelik süre aşıldı).

```c
#include <pthread.h>

typedef struct {
    int id;
    double balance;
    pthread_mutex_t lock;
} Account;

void transaction(Account *from, Account *to, double amount) {
    /* Kilitleri her zaman küçük id'den büyüğe doğru al. */
    Account *first  = (from->id < to->id) ? from : to;
    Account *second = (from->id < to->id) ? to : from;

    pthread_mutex_lock(&first->lock);
    pthread_mutex_lock(&second->lock);
    from->balance -= amount;   /* withdraw */
    to->balance   += amount;   /* deposit  */
    pthread_mutex_unlock(&second->lock);
    pthread_mutex_unlock(&first->lock);
}
```

6. (derste sözlü, 01:24:11) Banker algoritması örneğinde (Bölüm 10.5 tablosu, `Available` = (3, 3, 2)) sistem güvenli durumda mı?

> **Çözüm:** Evet. ⟨P1, P3, P4, P0, P2⟩ güvenli bir sıradır. `Work` sırasıyla (5, 3, 2), (7, 4, 3), (7, 4, 5), (7, 5, 5), (10, 5, 7) olur. Slayttaki ⟨P1, P3, P4, P2, P0⟩ sırası da güvenlidir.

7. (derste sözlü, 01:26:36-01:28:03) P1 (1, 0, 2) isterse karşılanır mı? Ardından P4 (3, 3, 0) isterse? Ardından P0 (0, 2, 0) isterse?

> **Çözüm:** P1'in isteği karşılanır: `Available` (2, 3, 0) olur ve ⟨P1, P3, P4, P0, P2⟩ hâlâ güvenlidir. P4'ün isteği `Need` sınırı içindedir ama A'dan yalnız 2 boş kopya vardır; P4 bekler. P0'ın isteği hem `Need` hem `Available` sınırı içindedir. Ama karşılanırsa `Available` = (2, 1, 0) olur ve hiçbir sürecin ihtiyacı karşılanamaz; durum güvensizdir. İstek reddedilir, P0 bekler. Derste bu son adım tamamlanmadı; sonuç Python ile doğrulandı.

8. (derste sözlü, 01:33:11) Tespit örneğinde (Bölüm 11.3 tablosu, `Available` = (0, 0, 0)) deadlock var mı? P2'nin isteği (0, 0, 1) olursa ne olur?

> **Çözüm:** İlk durumda deadlock yoktur: ⟨P0, P2, P3, P1, P4⟩ sırasıyla hepsi biter. P2'nin isteği (0, 0, 1) olursa yalnız P0 bitebilir; eldeki (0, 1, 0) ile kimsenin isteği karşılanamaz. P1, P2, P3 ve P4 deadlock'tadır.

Dersin Toparlaması

Bu derste ölümcül kilitlenme baştan sona ele alındı. Kaynakların istek, kullanım ve bırakma döngüsüyle başlanıp deadlock'ın dört gerekli koşulu görüldü: karşılıklı dışlama, tut ve bekle, kesme yok ve döngüsel bekleme. Bu koşulların dördü birden olmadıkça deadlock oluşmaz. Muteksleri ters sırayla alan iki thread örneği, bu dört koşulun gerçek bir programda nasıl bir araya geldiğini gösterdi. Kaynak atama çizgesinde döngü yoksa deadlock yoktur; döngü varsa ve döngüdeki kaynaklar tek kopyalıysa deadlock vardır; çok kopyalı kaynaklarda ise bir güvenli çalışma sırası aranmalıdır.

Ardından deadlock ile başa çıkmanın yolları incelendi. Önleme, dört koşuldan birini kırmaya çalışır ama her yöntem verim, açlık ya da taşınabilirlik sorunu getirir. Kaçınma, süreçlerin en fazla ihtiyaçlarını önceden bilerek sistemi hep güvenli durumda tutmaya çalışır; tek kopyalı kaynaklarda talep kenarlı RAG algoritması, çok kopyalı kaynaklarda banker algoritması kullanılır. Tespit için tek kopyalı kaynaklarda bekleme çizgesinde döngü aranır, çok kopyalı kaynaklarda banker'a benzer bir algoritma çalıştırılır. Kurtarmada kurban seçimi, geri alma ve açlık sorunları yüzünden modern işletim sistemleri bu işi genelde kullanıcıya bırakır. Hoca, bir sonraki derste bellek yönetimi (memory management) konusuna geçileceğini söyledi.

Not: İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.